

Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №9

із дисципліни «**Технології розроблення програмного забезпечення**»
Взаємодія компонентів системи.

Виконав:

студент групи ІА–34

Оборський Дмитро Вікторович

Тема: Взаємодія компонентів системи.

Мета: Вивчити види взаємодії додатків (Client-Server, Peer-to-Peer, Service-oriented Architecture), та реалізувати в проєктованій системі одну із архітектур.

14. **Архіватор** (strategy, adapter, factory method, facade, visitor, p2p)

Архіватор повинен являти собою візуальний додаток з можливістю створення і редагування архівів різного типу (.tar.gz, .zip, .rar, .ace) – додавання/ видалення файлів / папок, редагування метаданих (по можливості), перевірка checksum архівів, тестування архівів на наявність пошкоджень, розбиття архівів на частини.

Теоретичні відомості

Клієнт-серверні додатки являють собою найпростіший варіант розподілених додатків, де виділяється два види додатків: клієнти (представляють додаток користувачеві) і сервери (використовується для зберігання і обробки даних). Розрізняють тонкі клієнти і товсті клієнти.

Тонкий клієнт – клієнт, який повністю всі операції (або більшість, пов'язаних з логікою роботи програми) передає для обробки на сервер, а сам зберігає лише візуальне уявлення одержуваних від сервера відповідей. Грубо кажучи, тонкий клієнт – набір форм відображення і канал зв'язку з сервером. Прикладом тонкого клієнта є класичні Web-застосунки.

У такому варіанті використання майже все навантаження лягає на сервер або групу серверів.

Перевагою таких моделей є простота розгортання, тому що оновлювати потрібно лише сервери і в результаті клієнти з наступними запитами автоматично будуть працювати з оновленою системою.

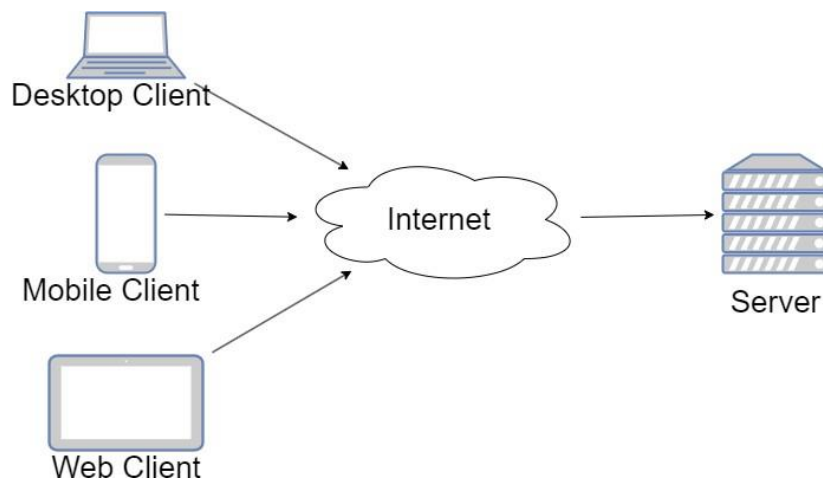


Рисунок 9.1. Клієнт-серверна архітектура

Товстий клієнт – антипод тонкого клієнта, більшість логіки обробки даних містить на стороні клієнта. Це сильно розвантажує сервер. Сервер в таких випадках зазвичай працює лише як точка доступу до деякого іншого ресурсу (наприклад, бази даних) або сполучна ланка з іншими клієнтськими комп'ютерами. Перевагою такого підходу є менші вимоги до серверної частини. Також перевагою, при певному підході до реалізації, є можливість працювати клієнтам без тимчасового доступу до серверу. Прикладом товстого клієнта можна назвати мобільні застосунки, або десктоп застосунки. Наприклад, Evernote, Viber, MS Outlook, комп'ютерні антивіруси, ігри, що потребують інсталяції (The Sims, GTA, ...) та інші.

Проміжним варіантом можна назвати SPA (Single Page Application) – це товсті Web-клієнти, які при старті кожен раз завантажуються з сервера, а надалі працюють з сервером через web-API. З одного боку більшу частину логіки вони відпрацьовують на клієнтській стороні, за рахунок чого зменшується серверне навантаження. Також оновлення простіше ніж для товстих клієнтів. Але такі застосунки не працюють, якщо сервер не доступний.

Клієнт-серверна взаємодія, як правило, організовується за допомогою 3-х рівневої структури: клієнтська частина, загальна частина, серверна частина.

Оскільки велика частина даних загальна (класи, використовувані системою), їх прийнято виносити в загальну частину (middleware) системи.

Клієнтська частина містить візуальне відображення і логіку обробки дії

користувача; код для встановлення сеансу зв'язку з сервером і виконання відповідних викликів.

Серверна частина містить основну логіку роботи програми (бізнес-логіку) або ту її частину, яка відповідає зберіганню або обміну даними між клієнтом і сервером або клієнтами.

1.1.1. Peer-to-Peer архітектура

Peer-to-Peer (P2P) архітектура – це модель мережевої взаємодії, в якій кожен вузол (комп'ютер або пристрій) є одночасно клієнтом і сервером. У цій архітектурі всі вузли мають рівні права та можливості для обміну даними,

ресурсами або виконання завдань. На відміну від клієнт-серверної моделі, де є чітке розділення на клієнти й сервери, P2P-мережа дозволяє учасникам взаємодіяти безпосередньо, без необхідності в централізованому сервері.

Основними принципами P2P-архітектури є:

- Децентралізація – відсутність центрального сервера, що зменшує залежність від одного вузла, підвищуючи стійкість мережі до збоїв і атак.
- Рівноправність вузлів – кожен вузол може виконувати одночасно функції клієнта (отримувати ресурси) і сервера (надавати ресурси).
- Розподіл ресурсів – вузли надають доступ до своїх власних ресурсів, таких як обчислювальна потужність, дисковий простір або файли.

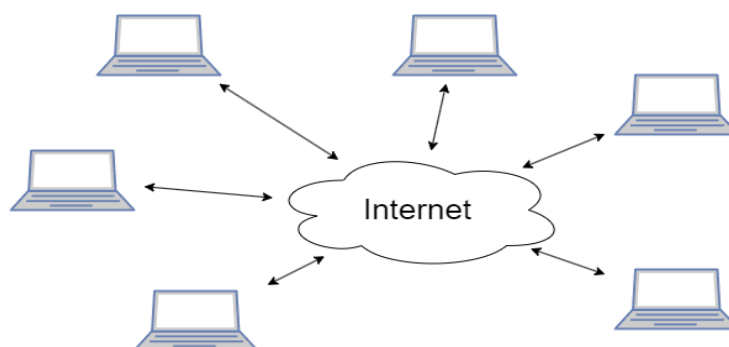


Рисунок 2. Peer-to-Peer архітектура

Основними сферами де peer-to-peer архітектура знайшла широке

застосування є файлообмінники (BitTorrent), криптовалюти та інші блокчейн-технології, інтернет телефонія та відеоконференції (Skype, Zoom), розподілені обчислення (SETI@home, BOINC).

До основних проблемних зон можна віднести безпеку, синхронізацію даних та пошук ресурсів. Через централізацію складно контролювати дані, які передаються. Ефективність пошуку даних знижується зі збільшенням кількості вузлів у мережі і для підвищення ефективності пошуку потрібно застосовувати спеціальні алгоритми.

1.1.2. Сервіс-орієнтована архітектура

Сервіс-орієнтована архітектура (SOA, англ. service-oriented architecture) – модульний підхід до розробки програмного забезпечення, заснований на використанні розподілених, слабо пов'язаних (англ. Loose coupling) сервісів або служб, оснащених стандартизованими інтерфейсами для взаємодії за стандартизованими протоколами [11].

Історично сервіс-орієнтована архітектура появилась як альтернатива монолітній архітектурі, в якій вся система розроблялася та розгорталася як одне ціле.

Програмні комплекси, розроблені відповідно до сервіс-орієнтованою архітектурою, зазвичай реалізуються як набір веб-служб (або веб-сервісів), які, як правило, взаємодіють по HTTP з використанням SOAP або REST. Ці служби надають певні бізнес-функції, наприклад, отримання інформації про наявність матеріалів на складі.

Сервіси взаємодіють між собою тільки за рахунок обміну повідомленнями, без створення спеціальних інтеграцій для доступу до однієї інформації, наприклад, до однієї бази даних.

Сервіси також можуть бути реалізовані як обгортки навколо застарілої системи. Це робиться для зменшення вартості переробки системи, а також спрощення інтеграції існуючих монолітних систем в нову архітектуру.

Згідно SOA сервіси реєструються на спеціальних сервісах і будь-яка команда розробників, якій потрібен доступ може знайти їх та використовувати.

Часто реалізація SOA покладається на використання централізованого програмного компонента для обміну даними – шину даних (Enterprise Service Bus).

Мікросервісна архітектура є подальшим розвитком сервіс-орієнтованої архітектури з використанням нових напрацювань у інформаційних технологіях.

1.1.3. Мікро-сервісна архітектура.

Сама назва дає зрозуміти, що мікро-сервісна архітектура є підходом до створення серверного додатку як набору малих служб [11]. Це означає, що архітектура мікро-сервісів головним чином орієнтована на серверну частину, не дивлячись на те, що цей підхід так само використовується для зовнішнього інтерфейсу, де кожна служба виконується в своєму процесі і взаємодіє з іншими службами за такими протоколами, як HTTP/HTTPS, WebSockets чи AMQP. Кожен мікросервіс реалізує специфічні можливості в предметній області і свою бізнес-логіку в рамках конкретного обмеженого контексту, повинна розроблятися автономно і розвертатися незалежно.

Визначення мікросервісів із книги Іраклі Надарейшвілі, Ронні Мітра, Метта Макларті та Майка Амундсена (О'Рейлі) «Архітектура мікросервісів»:

«Мікросервіс – це компонент із чітко визначеними межами, який можна розгорнути незалежно, і підтримує взаємодію за допомогою зв'язку на основі повідомлень. Архітектура мікросервісів – це стиль розробки високоавтоматизованих систем програмного забезпечення, що легко розвивати та яке складається з мікросервісів, орієнтованих на певні можливості».

Мікросервіси забезпечують чудові можливості супроводження в величезних комплексних системах з високою масштабуємістю за рахунок створення додатків, заснованих на множині незалежно розгортуючих служб з автономними життєвими циклами.

Хід роботи

1. В моїй роботі я використовую Client-Server архітектуру за участю JDBC, HikariCP та Docker. Для цього спочатку підіймаю БД у DataSourceFactory:

```
1 package com.beowulf.core.db;
2
3 import com.zaxxer.hikari.HikariConfig;
4 import com.zaxxer.hikari.HikariDataSource;
5
6 import javax.sql.DataSource;
7
8 You, 6 days ago | 1 author (You)
9 public final class DataSourceFactory {
10     private static HikariDataSource dataSource;
11
12     private DataSourceFactory() {
13     }
14
15     public static DataSource getDataSource() {
16         if (dataSource == null) {
17             HikariConfig config = new HikariConfig();
18
19             String url = System.getenv("BEOWULF_DB_URL");
20             if (url == null || url.isBlank()) {
21                 url = "jdbc:postgresql://localhost:5432/beowulf_db";
22             }
23             config.setJdbcUrl(url);
24
25             String user = System.getenv("BEOWULF_DB_USER");
26             if (user == null || user.isBlank()) {
27                 user = "beowulf";
28             }
29             config.setUsername(user);
30
31             String pass = System.getenv("BEOWULF_DB_PASS");
32             if (pass == null || pass.isBlank()) {
33                 pass = "beowulf";
34             }
35             config.setPassword(pass);
36
37             config.setMaximumPoolSize(maxPoolSize: 10);
38             config.setMinimumIdle(minIdle: 1);
39             config.setPoolName(poolName: "BeowulfHikariPool");
40
41             dataSource = new HikariDataSource(config);
42         }
43         return dataSource;
44     }
45
46     public static void close() {
47         if (dataSource != null) {
48             dataSource.close();
49         }
50     }
51 }
```

Рис. 1 DataSourceFactory

2. Для маніпуляції даними на сервері я також створюю ArchivePersistenceService та ArchiveLogService

```
You, 1 hour ago | 1 author (You)
1 package com.beowulf.core.service;
2
3 import com.beowulf.core.db.DataSourceFactory;
4 import com.beowulf.core.model.ArchiveLog;
5 import com.beowulf.core.model.ArchivePart;
6
7 import javax.sql.DataSource;
8 import java.sql.*;
9 import java.time.ZoneOffset;
10 import java.util.ArrayList;
11 import java.util.List;
12 import java.util.UUID;
13
You, 3 days ago | 1 author (You)
14 public class ArchiveLogService {
15
16     private final DataSource dataSource;
17
18     public ArchiveLogService() {
19         this.dataSource = DataSourceFactory.getDataSource();
20     }
21
22     public List<ArchivePart> findArchiveParts(UUID archiveId) {
23         String sql = """
24             SELECT id, archive_id, part_index, path, size_bytes, created_at
25             FROM archive_part
26             WHERE archive_id = ?
27             ORDER BY part_index
28             """;
29
30         List<ArchivePart> parts = new ArrayList<>();
31
32         try (Connection connection = dataSource.getConnection();
33             PreparedStatement preparedStatement = connection.prepareStatement(sql)) {
34             preparedStatement.setObject(1, archiveId);
35
36             try (ResultSet resultSet = preparedStatement.executeQuery()) {
37                 while (resultSet.next()) {
38                     ArchivePart part = new ArchivePart();
39                     part.setPartIndex(resultSet.getInt("part_index"));
40                     part.setPath(resultSet.getString("path"));
41                     part.setSizeBytes(resultSet.getLong("size_bytes"));
42                     parts.add(part);
43                 }
44             }
45         } catch (SQLException error) {
46             throw new RuntimeException("Failed to load archive parts", error);
47         }
48
49         return parts;
50     }
51
52     public List<ArchiveLog> findRecentLogs(UUID userId, int limit) {
53         String sql = """
54             SELECT
55                 al.id AS log_id,
56                 al.user_id AS user_id,
57                 al.archive_id AS archive_id,
58                 al.operation,
59                 al.status,
60                 al.target_path,
61                 al.duration_ms,
62                 al.created_at,
63
64                 a.path AS archive_path,
65                 a.format AS format,
66                 a.compression AS compression,
67                 a.size_bytes AS size_bytes,
68
69                 -- NEW: arperaru no archive_part
70                 COALESCE(parts.parts_count, 0) AS split_parts_count,
71                 COALESCE(parts.total_size, 0) AS split_total_size,
72                 parts.first_part_path AS split_first_path
73             FROM archive_log al
74             JOIN archive a ON a.id = al.archive_id
75             LEFT JOIN (
76                 SELECT
77                     ap.archive_id,
78                     COUNT(*) AS parts_count,
79                     SUM(ap.size_bytes) AS total_size,
80                     MIN(ap.path) AS first_part_path
81                 FROM archive_part ap
82                 GROUP BY ap.archive_id
83             ) parts ON parts.archive_id = a.id
84             WHERE al.user_id = ?
85             ORDER BY al.created_at DESC
86             LIMIT ?
87             """;
```

Рис. 2 ArchiveLogService частина 1


```

14 public class ArchiveLogService {
15     public List<ArchiveLog> findArchiveLogs(UUID userId, int limit) {
16         a format AS format,
17         a compression AS compression,
18         a size_bytes AS size_bytes,
19
20         -- NEW: aggregate no archive_part
21         COALESCE(parts.parts_count, 0) AS split_parts_count,
22         COALESCE(parts.total_size, 0) AS split_total_size,
23         parts.first_part_path AS split_first_path
24     FROM archive_log al
25     JOIN archive a ON a.id = al.archive_id
26     LEFT JOIN (
27         SELECT
28             ap.archive_id,
29             COUNT(*) AS parts_count,
30             SUM(ap.size_bytes) AS total_size,
31             MIN(ap.path) AS first_part_path
32         FROM archive_part ap
33         GROUP BY ap.archive_id
34     ) parts ON parts.archive_id = a.id
35     WHERE al.user_id = ?
36     ORDER BY al.created_at DESC
37     LIMIT ?
38     """
39
40     List<ArchiveLog> logs = new ArrayList<>();
41
42     try (Connection connection = dataSource.getConnection();
43         PreparedStatement preparedStatement = connection.prepareStatement(sql)) {
44         preparedStatement.setString(1, userId);
45         preparedStatement.setInt(2, limit);
46
47         try (ResultSet resultSet = preparedStatement.executeQuery()) {
48             while (resultSet.next()) {
49                 ArchiveLog log = new ArchiveLog();
50
51                 log.setId(resultSet.getLong("log_id"));
52                 log.setUserid(resultSet.getLong("user_id"));
53                 log.setArchiveId(resultSet.getLong("archive_id"));
54                 log.setOperation(resultSet.getString("operation"));
55                 log.setStatus(resultSet.getString("status"));
56                 log.setTargetPath(resultSet.getString("target_path"));
57                 log.setDuration(resultSet.getLong("duration_ms"));
58
59                 Timestamp ts = resultSet.getTimestamp("created_at");
60                 if (ts != null) {
61                     log.setCreatedAt(ts.toInstant().atOffset(ZoneOffset.UTC));
62                 }
63
64                 log.setArchivePath(resultSet.getString("archive_path"));
65                 log.setFormat(resultSet.getString("format"));
66                 log.setCompression(resultSet.getString("compression"));
67                 log.setSizeBytes(resultSet.getLong("size_bytes"));
68
69                 long partsCount = resultSet.getLong("split_parts_count");
70                 if (resultSet.isNull()) {
71                     log.setSplitPartsCount(0);
72                 } else {
73                     log.setSplitPartsCount(partsCount);
74                 }
75
76                 long totalSize = resultSet.getLong("split_total_size");
77                 if (resultSet.isNull()) {
78                     log.setSplitTotalSize(0);
79                 } else {
80                     log.setSplitTotalSize(totalSize);
81                 }
82
83                 log.setSplitFirstPath(resultSet.getString("split_first_path"));
84
85                 logs.add(log);
86             }
87         }
88     } catch (SQLException error) {
89         throw new RuntimeException("Failed to load archive logs", error);
90     }
91
92     return logs;
93 }

```

Рис. 3 ArchiveLogService часть 2

```

1 package com.bonnullif.core.service;
2
3 import com.bonnullif.core.db.DataSourceFactory;
4 import com.bonnullif.core.model.ArchivePart;
5 import com.bonnullif.core.user.ApiUser;
6 import com.bonnullif.core.visitor.ArchiveOperation;
7
8 import javax.sql.DataSource;
9 import java.sql.*;
10 import java.util.*;
11 import java.util.UUID;
12
13 public class ArchivePersistenceService {
14     private final DataSource dataSource;
15
16     public ArchivePersistenceService() {
17         this.dataSource = DataSourceFactory.getDataSource();
18     }
19
20     /**
21      * Persist an archive operation.
22      *
23      * @param ctx the ArchiveOperation object to persist
24      */
25     public void persistOperation(ArchiveOperation ctx) {
26         Connection connection = null;
27         try {
28             connection = dataSource.getConnection();
29             connection.setAutoCommit(false);
30
31             ApiUser user = ctx.getUser();
32             updateUser(connection, user);
33
34             UUID checksumId = ctx.getChecksumId();
35             if (checksumId == null) {
36                 checksumId = UUID.randomUUID();
37             }
38
39             String checksumValue = ctx.getChecksumValue();
40             if (checksumValue == null) {
41                 checksumValue = "";
42             }
43
44             insertChecksum(connection, checksumId,
45                 ctx.getChecksumType(),
46                 checksumValue);
47
48             ctx.setChecksumId(checksumId);
49             ctx.setChecksumValue(checksumValue);
50
51             UUID archiveId = ctx.getArchiveId();
52             if (archiveId == null) {
53                 archiveId = UUID.randomUUID();
54             }
55             insertArchive(connection,
56                 archiveId,
57                 user.getId(),
58                 checksumId,
59                 ctx.getFormat(),
60                 ctx.getCompression(),
61                 ctx.getArchivePath(),
62                 ctx.getSizeBytes());
63         } else {
64             updateArchive(connection,
65                 archiveId,
66                 user.getId(),
67                 checksumId,
68                 ctx.getFormat(),
69                 ctx.getCompression(),
70                 ctx.getArchivePath(),
71                 ctx.getSizeBytes());
72         }
73         ctx.setArchiveId(archiveId);
74
75         insertArchiveLog(connection,
76             user.getId(),
77             archiveId,
78             ctx.getOperation(),
79             ctx.getStatus(),
80             ctx.getTargetPath(),
81             ctx.getDurationMs());
82
83         if (ctx.getParts() != null && !ctx.getParts().isEmpty()) {
84             saveArchiveParts(archiveId, ctx.getParts());
85         }
86     }
87 }

```

Рис. 4 ArchivePersistenceService часть 1

```

1 package com.bowulf.core.service;
2
3 import com.bowulf.core.db.DataSourceFactory;
4 import com.bowulf.core.model.ArchivePart;
5 import com.bowulf.core.user.AppUser;
6 import com.bowulf.core.visitor.ArchiveOperation;
7
8 import java.sql.DataSource;
9 import java.sql.*;
10 import java.util.List;
11 import java.util.UUID;
12
13 public class ArchivePersistenceService {
14
15     private final DataSource dataSource;
16
17     public ArchivePersistenceService() {
18         this.dataSource = DataSourceFactory.getDataSource();
19     }
20
21     /**
22      * Persist an archive operation.
23      *
24      * @param ctx the ArchiveOperation object to persist
25      */
26
27     public void persistOperation(ArchiveOperation ctx) {
28         Connection connection = null;
29         try {
30             connection = dataSource.getConnection();
31             connection.setAutoCommit(false);
32
33             AppUser user = ctx.getUser();
34             upsertUser(connection, user);
35
36             UUID checksumId = ctx.getChecksumId();
37             if (checksumId == null) {
38                 checksumId = UUID.randomUUID();
39             }
40
41             String checksumValue = ctx.getChecksumValue();
42             if (checksumValue == null) {
43                 checksumValue = "";
44             }
45
46             insertChecksum(connection, checksumId,
47                 ctx.getChecksumType(),
48                 checksumValue);
49
50             ctx.setChecksumId(checksumId);
51             ctx.setChecksumValue(checksumValue);
52
53             UUID archiveId = ctx.getArchiveId();
54             if (archiveId == null) {
55                 archiveId = UUID.randomUUID();
56             }
57             insertArchive(connection,
58                 archiveId,
59                 user.getId(),
60                 checksumId,
61                 ctx.getFormat(),
62                 ctx.getCompression(),
63                 ctx.getArchivePath(),
64                 ctx.getSizeBytes());
65             } else {
66                 updateArchive(connection,
67                     archiveId,
68                     user.getId(),
69                     checksumId,
70                     ctx.getFormat(),
71                     ctx.getCompression(),
72                     ctx.getArchivePath(),
73                     ctx.getSizeBytes());
74             }
75             ctx.setArchiveId(archiveId);
76
77             insertArchiveLog(connection,
78                 user.getId(),
79                 archiveId,
80                 ctx.getOperation(),
81                 ctx.getStatus(),
82                 ctx.getTargetPath(),
83                 ctx.getStorageMeta());
84
85             if (ctx.getPartId() != null && !ctx.getParts().isEmpty()) {
86                 saveArchiveParts(archiveId, ctx.getParts());
87             }
88         }

```

Рис. 5 ArchivePersistenceService часть 2

```

13 public class ArchivePersistenceService {
14
15     public void persistOperation(ArchiveOperation ctx) {
16
17         connection.commit();
18
19         connection.commit();
20
21         } catch (SQLException error) {
22             safeRollback(connection);
23             throw new RuntimeException("Failed to persist archive operation", error);
24         } finally {
25             safeClose(connection);
26         }
27     }
28
29     /**
30      * Insert a user.
31      *
32      * @param connection the database connection
33      * @param user the AppUser object to insert
34      */
35
36     private void upsertUser(Connection connection, AppUser user) throws SQLException {
37         String sql = "
38             INSERT INTO app_user (id, username)
39             VALUES (?, ?)
40             ON CONFLICT (id) DO UPDATE
41             SET username = EXCLUDED.username
42             ";
43
44         try (PreparedStatement preparedStatement = connection.prepareStatement(sql)) {
45             preparedStatement.setObject(1, user.getId());
46             preparedStatement.setString(2, user.getUsername());
47             preparedStatement.executeUpdate();
48         }
49     }
50
51     /**
52      * Insert a checksum.
53      *
54      * @param connection the database connection
55      * @param id the ID of the checksum
56      * @param type the type of the checksum
57      * @param value the value of the checksum
58      */
59
60     private void insertChecksum(Connection connection,
61         UUID id,
62         String type,
63         String value) throws SQLException {
64         if (type == null || type.isBlank()) {
65             type = "unknown";
66         }
67         if (value == null || value.isBlank()) {
68             value = "N/A";
69         }
70
71         String sql = "
72             INSERT INTO checksum (id, type, value)
73             VALUES (?, ?, ?)
74             ";
75
76         try (PreparedStatement preparedStatement = connection.prepareStatement(sql)) {
77             preparedStatement.setObject(1, id);
78             preparedStatement.setString(2, type);
79             preparedStatement.setString(3, value);
80             preparedStatement.executeUpdate();
81         }
82     }
83
84     /**
85      * Insert an archive.
86      *
87      * @param connection the database connection
88      * @param archiveId the ID of the archive
89      * @param userId the ID of the user
90      * @param checksumId the ID of the checksum
91      * @param format the format of the archive
92      * @param compression the compression of the archive
93      * @param path the path of the archive
94      * @param sizeBytes the size of the archive
95      */
96
97     private void insertArchive(Connection connection,

```

Рис. 6 ArchivePersistenceService часть 3

```

31 public class ArchivePersistenceService {
32     public void persistOperation(ArchiveOperation op) {
33         //
34         //
35         connection.commit();
36         connection.commit();
37         } catch (SQLException error) {
38             safeRollback(connection);
39             throw new RuntimeException("Failed to persist archive operation", error);
40         } finally {
41             safeClose(connection);
42         }
43     }
44     /**
45      * Insert a user.
46      *
47      * @param connection the database connection
48      * @param user the AppUser object to insert
49      */
50     private void insertUser(Connection connection, AppUser user) throws SQLException {
51         String sql = "
52             INSERT INTO app_user (id, username)
53             VALUES (?, ?)
54             ON CONFLICT (id) DO UPDATE
55             SET username = EXCLUDED.username
56             ";
57         try (PreparedStatement preparedStatement = connection.prepareStatement(sql)) {
58             preparedStatement.setString(1, user.getId());
59             preparedStatement.setString(2, user.getUsername());
60             preparedStatement.executeUpdate();
61         }
62     }
63     /**
64      * Insert a checksum.
65      *
66      * @param connection the database connection
67      * @param id the ID of the checksum
68      * @param type the type of the checksum
69      * @param value the value of the checksum
70      */
71     private void insertChecksum(Connection connection,
72                                UUID id,
73                                String type,
74                                String value) throws SQLException {
75         if (type == null || type.isEmpty()) {
76             type = "UNKNOWN";
77         }
78         if (value == null || value.isEmpty()) {
79             value = "N/A";
80         }
81         String sql = "
82             INSERT INTO checksum (id, type, value)
83             VALUES (?, ?, ?)
84             ";
85         try (PreparedStatement preparedStatement = connection.prepareStatement(sql)) {
86             preparedStatement.setString(1, id);
87             preparedStatement.setString(2, type);
88             preparedStatement.setString(3, value);
89             preparedStatement.executeUpdate();
90         }
91     }
92     /**
93      * Insert an archive.
94      *
95      * @param connection the database connection
96      * @param archiveId the ID of the archive
97      * @param userId the ID of the user
98      * @param checksumId the ID of the checksum
99      * @param format the format of the archive
100      * @param compression the compression of the archive
101      * @param path the path of the archive
102      * @param sizeBytes the size of the archive
103      */
104     private void insertArchive(Connection connection,
105                                UUID archiveId,
106                                UUID userId,
107                                UUID checksumId,
108                                String format,
109                                String compression,
110                                String path,
111                                Long sizeBytes) throws SQLException {

```

Рис. 7 ArchivePersistenceService частина 4

3. В якості клієнта в мене виступають основні класи інтерфейсів BeowulfCLI та BeowulfGUI. Враховуючи, що я згадував їх у попередніх роботах, прикріплю уривки

```

18 import java.util.List;
19
20 @SuppressWarnings("unchecked")
21 public class BeowulfCLI {
22     private static final ArchiveFactory factory = new ArchiveFactory();
23
24     private static AppUserProvider identityProvider;
25     private static ArchivePersistenceService persistenceService;
26     private static ArchiveVisitor persistenceVisitor;
27     private static ArchiveLoggerService loggerService;
28
29     public static void main(String[] args) {
30         //
31         //
32         identityProvider = new AppUserProvider();
33         persistenceService = new ArchivePersistenceService();
34         persistenceVisitor = new ArchiveVisitor(persistenceService);
35         loggerService = new ArchiveLoggerService();
36
37         if (args.length == 1) {
38             printUsage();
39             return;
40         }
41
42         String command = args[0];
43
44         try {
45             switch (command) {
46                 case "compress" -> handleCompress(args);
47                 case "decompress" -> handleDecompress(args);
48                 case "help" -> printUsage();
49                 default -> {
50                     System.out.println("Unknown command: " + command);
51                     printUsage();
52                 }
53             }
54         } catch (Exception error) {
55             System.err.println("Error: " + error.getMessage());
56             error.printStackTrace();
57             System.exit(1);
58         }
59     }
60
61     private static void handleCompress(String[] args) throws IOException {
62         if (args.length != 3) {
63             System.err.println("Invalid usage for compress.");
64             System.err.println("Usage: beowulf compress <sourceDir> <targetArchive>");
65             return;
66         }
67
68         Path sourceDir = Paths.get(args[1]);
69         Path targetArchive = Paths.get(args[2]);
70
71         Archive baseArchive = factory.getArchive(targetArchive);
72         Archive archive = new ArchiveWrapper(
73             baseArchive,
74             identityProvider,
75             persistenceVisitor);
76
77         System.out.println("Compressing using " + archive.getName());
78         archive.compress(sourceDir, targetArchive);
79         System.out.println("Done: " + targetArchive);
80     }
81
82     private static void handleDecompress(String[] args) throws IOException {
83         if (args.length != 3) {
84             System.err.println("Invalid usage for decompress.");
85             System.err.println("Usage: beowulf decompress <archive> <targetDir>");
86             return;
87         }
88
89         Path archive = Paths.get(args[1]);
90         Path targetDir = Paths.get(args[2]);
91
92         Archive baseArchive = factory.getArchive(archive);
93         Archive archive = new ArchiveWrapper(
94             baseArchive,
95             identityProvider,
96             persistenceVisitor);

```

Рис. 8 BeowulfCLI

```

54 You, 3 days ago | 1 author (You)
55 public class BeowulfGUI extends Application {
56
57     private AppUserService identityProvider;
58     private ArchivePersistenceService persistenceService;
59     private ArchiveLogService logQueryService;
60     private ArchiveVisitor persistVisitor;
61     private ArchiveFactory archiverFactory;
62     private ArchiveEditService editService;
63
64     private Stage primaryStage;
65
66     private final ObservableList<LogRow> logRows = FXCollections.observableArrayList();
67
68     private static final DateTimeFormatter TABLE_TIME_FORMAT = DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm:ss");
69     private static final DateTimeFormatter DETAIL_TIME_FORMAT = DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm:ss");
70     private static final Locale ENGLISH = Locale.ENGLISH;
71
72     private TextField editArchiveField;
73     private TreeView<FileNode> editTreeView;
74     private ProgressBar editProgressBar;
75     private Label editStatusLabel;
76
77     private EditSession currentSession;
78
79     private static final DataFormat EDIT_INTERNAL_MOVE = new DataFormat("beowulf/edit-internal-move");
80     private Path dragSourcePath;
81
82     private static final String[] ARCHIVE_EXTENSIONS = {
83         ".zip", ".tar.gz", ".tgz", ".rar", ".7z", ".tar", ".ace"
84     };
85
86     private Image folderClosedIcon;
87     private Image folderOpenIcon;
88     private Image fileIcon;
89     private Image archiveIcon;
90
91     @Override
92     public void start(Stage primaryStage) {
93         this.primaryStage = primaryStage;
94
95         initCore();
96         loadIcons();
97
98         TabPane tabPane = new TabPane();
99         tabPane.setTabMinWidth(130);
100
101         tabPane.getTabs().addAll(
102             buildCompressTab(primaryStage),
103             buildDecompressTab(primaryStage),
104             buildEditTab(primaryStage),
105             buildLogsTab();
106
107         BorderPane root = new BorderPane(tabPane);
108         root.setPadding(new Insets(topRightBottomLeft: 10));
109
110         Scene scene = new Scene(root, width: 1150, height: 720);
111         root.setStyleClass("root");
112         -fx-background-color: #f2f3f5;
113         -fx-font-family: "Segoe UI", "Roboto", sans-serif;
114         -fx-font-size: 16px;
115         ***);
116
117         primaryStage.setTitle("Beowulf Archiver");
118         primaryStage.setScene(scene);
119         primaryStage.setMinWidth(900);
120         primaryStage.setMinHeight(600);
121         primaryStage.show();
122
123         reloadLogs();
124     }
125
126     private void initCore() {
127         OMigrations migrate();
128         this.identityProvider = new AppUserService();
129         this.persistenceService = new ArchivePersistenceService();
130         this.logQueryService = new ArchiveLogService();
131         this.persistVisitor = new ArchiveVisitor(persistenceService);
132         this.archiverFactory = new ArchiveFactory();
133         this.editService = new ArchiveEditService(archiverFactory, identityProvider, persistenceService);
134     }
135
136     private void loadIcons() {
137         folderClosedIcon = loadIcon("icons/folder-closed.png");
138     }

```

Рис. 9 BeowulfGUI

4. Розроблюю діаграму класів

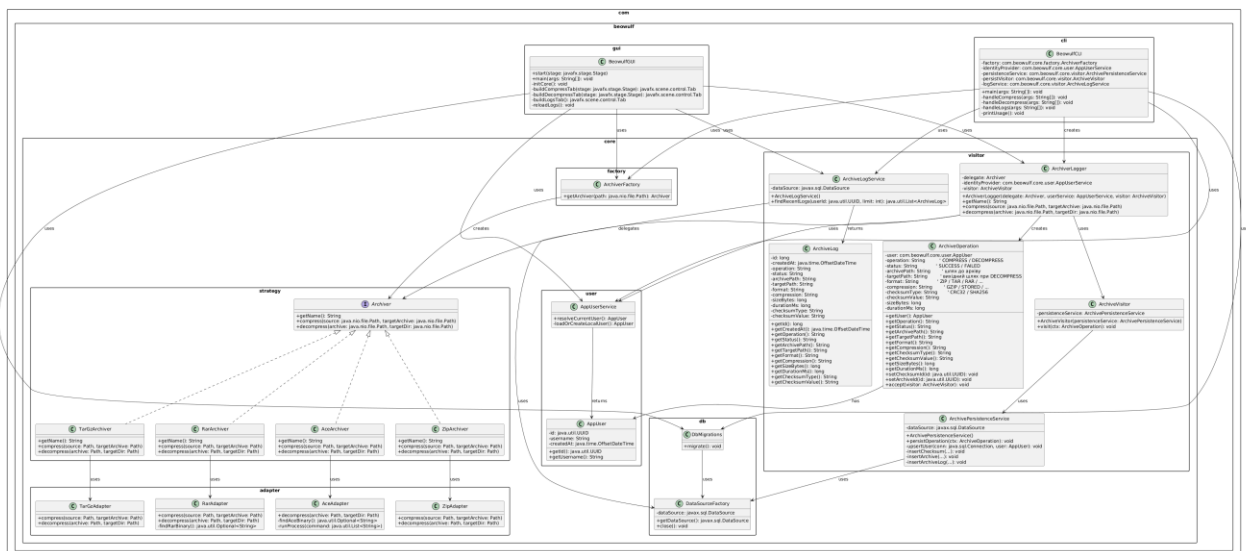


Рис. 4 Діаграма класів

Висновок

У ході цієї лабораторної роботи я закріпив теоретичні знання про моделі взаємодії компонентів програмних систем — клієнт-серверну архітектуру, Peer-to-Peer, SOA та мікросервісний підхід — і проаналізував їхні відмінності та сфери застосування. На практиці я реалізував клієнт-серверну взаємодію у своєму проєкті архіватора Beowulf, використавши JDBC, HikariCP та Docker для організації доступу до бази даних. Було створено серверну частину у вигляді сервісів ArchivePersistenceService та ArchiveLogService, а клієнтську частину представлено компонентами BeowulfCLI та BeowulfGUI. Також була розроблена діаграма класів, яка демонструє взаємозв'язки між клієнтом, сервером і загальними компонентами системи. Загалом робота показала, що правильне проєктування взаємодії між компонентами забезпечує масштабованість, надійність та зручність супроводу програмної системи.

Контрольні запитання

1. Що таке клієнт-серверна архітектура?

(Це модель взаємодії, у якій клієнт відправляє запити, а сервер обробляє їх і повертає результати. Клієнт відповідає за інтерфейс та взаємодію з користувачем, сервер — за логіку і зберігання даних)

2. Розкажіть про сервіс-орієнтовану архітектуру.

(SOA — це підхід, де система складається з незалежних сервісів зі стандартизованими інтерфейсами. Кожен сервіс виконує чітко визначену бізнес-функцію і взаємодіє з іншими сервісами через обмін повідомленнями)

3. Якими принципами керується SOA?

(Основні принципи: слабка зв'язаність, повторне використання сервісів, стандартизовані інтерфейси, автономність, ізольованість бізнес-функцій, можливість незалежного оновлення сервісів)

4. Як між собою взаємодіють сервіси в SOA?

(Сервіси взаємодіють виключно через обмін повідомленнями — найчастіше HTTP(S) із SOAP або REST, не маючи прямого доступу до спільних ресурсів чи баз даних)

5. Як розробники взнають про існуючі сервіси і як робити до них запити?

(Сервіси реєструються у спеціальному реєстрі або каталозі (service registry), з якого розробники отримують інформацію про доступні API та їхні точки входу. Виклик сервісів здійснюється через стандартизовані HTTP-запити)

6. У чому полягають переваги та недоліки клієнт-серверної моделі?

- ✧ Переваги: просте розгортання, централізоване керування, контроль доступу, надійність.
- ✧ Недоліки: залежність від сервера, потреба у масштабуванні, можливі затримки при великому навантаженні.

7. У чому полягають переваги та недоліки однорангової моделі взаємодії?

- ✧ Переваги: децентралізація, відсутність єдиної точки відмови, висока масштабованість, ефективний розподіл ресурсів.
- ✧ Недоліки: складність у забезпеченні безпеки, складні алгоритми пошуку ресурсів, проблеми з узгодженням даних.

8. Що таке мікросервісна архітектура?

(Це стиль розробки, у якому система складається з набору невеликих незалежних сервісів, кожен з яких виконує одну бізнес-функцію, розгортається автономно і спілкується з іншими через легкі протоколи)

9. Які протоколи використовуються для обміну даними в мікросервісній архітектурі?

(HTTP/HTTPS (REST), WebSockets, gRPC, AMQP (RabbitMQ), Kafka-протоколи — залежно від характеру взаємодії)

10. Чи можна назвати підхід сервіс-орієнтованою архітектурою, коли ми в проєкті між шаром веб-контролерів та шаром доступу до даних реалізуємо шар бізнес-логіки у вигляді сервісів?

(Ні. Це просто трирівнева архітектура з шаром сервісів, але не SOA, оскільки такі “сервіси” не є незалежними, не мають власних інтерфейсів взаємодії, не розгортаються окремо і не обмінюються повідомленнями як ізольовані компоненти)